

Analysis

Cam Smithers

2026-06-30

Analysis

Packages

```
set.seed(157)
library(tidyverse)
library(glue)
library(zoo)
library(psych)
library(ggthemes)
library(viridis)
library(hrbrthemes)
library(nnet)
```

Functions Script

```
setwd('/Users/camsmithers/Desktop/Camalytics/NBA')
source('inR/Functions.R')
```

Analysis Cleaning

Data Sets

Loading in the data sets that I'll need for analysis.

```
setwd('/Users/camsmithers/Desktop/Camalytics/NBA')
team_box_scores <- readRDS(
  'Data-NBA/Current/team_box_scores_current.rds')
player_box_scores <- readRDS(
  'Data-NBA/Current/full_box_player_current.rds')
team_season_stats <- readRDS(
  'Data-NBA/Current/team_season_stats_current.rds')
player_season_stats <- readRDS(
  'Data-NBA/Current/player_season_stats_current.rds')
```

Applying a function I wrote that fixes some aesthetic issues with player names. This will occur with foreign players that have special characters or foreign characters in their name.

```
player_box_scores <- fixplayername(player_box_scores)
player_season_stats <- fixplayername(player_season_stats)
```

Outcomes and Small Metrics

Outcome I'm adding the points from the opposing team so I can create an outcome variable based on the margin of the outcome. This will be multinomial regression with four different classifications. Those are...

```
opponent_stats <- team_box_scores %>%
  select(gamedate, team, pts) %>%
  rename('opp_pts'='pts')

team_box_scores <- team_box_scores %>%
  left_join(opponent_stats, by = c('gamedate', 'opponent'='team'))

team_box_scores <- team_box_scores %>%
  mutate(
    ptsdiff = pts - opp_pts,
    twoleveloc = if_else(offrating > defrating, 1, 0),
    threeleveloc = case_when(
      ptsdiff >= 11 ~ 2,
      ptsdiff %in% 1:10 ~ 1,
      TRUE ~ 0),
    fourleveloc = case_when(
      ptsdiff <= -11 ~ 0,
      ptsdiff %in% -1:-10 ~ 1,
      ptsdiff %in% 1:10 ~ 2,
      ptsdiff >= 11 ~ 3)) %>%
  mutate(
    twoleveloc = factor(twoleveloc,
      levels = c(0, 1),
      labels = c('Loss', 'Win')),
    threeleveloc = factor(threeleveloc,
      levels = c(0, 1, 2),
      labels = c('Loss', 'Basic Win', 'Blowout Win')),
    fourleveloc = factor(fourleveloc,
      levels = c(0, 1, 2, 3),
      labels = c('Blowout Loss', 'Basic Loss',
        'Basic Win', 'Blowout Win')))
```

Conditional Outcomes I want to look at how missing a player affects the chance of a team to win a game; however, I have to determine which players are important for the outcome of the game. I'm going to look at few metrics and use those to determine a player's significance. For the different metrics, different players may end up being the most important player. Those metrics include...

1. Win Shares: `ws`
2. Value Over Replacement Player (VORP): `vorp`
3. Player Efficiency Rating: `player_effrtg`

I'm also going to create a variable that has a unique id for each best player by a given metric.

```
bp_by_ws <- best_player_metric(player_season_stats, ws, 20, 10) %>%
  rename('ws_name'='name') %>%
  unite('ws_team_id', c('year', 'team', 'ws_player_id'),
    sep = '_', remove = FALSE) %>%
  mutate(ws_team_id = paste0('ws_', ws_team_id)) %>%
  relocate(ws_team_id, .after = last_col())
bp_by_vorp <- best_player_metric(player_season_stats, vorp, 20, 10) %>%
  rename('vorp_name'='name') %>%
```

```

unite('vorp_team_id', c('year', 'team', 'vorp_player_id'),
      sep = '_', remove = FALSE) %>%
mutate(vorp_team_id = paste0('vorp_', vorp_team_id)) %>%
relocate(vorp_team_id, .after = last_col())
bp_by_per <- best_player_metric(player_season_stats, player_effrtg, 20, 10) %>%
rename('per_name'='name') %>%
unite('per_team_id', c('year', 'team', 'player_effrtg_player_id'),
      sep = '_', remove = FALSE) %>%
mutate(per_team_id = paste0('per_', per_team_id)) %>%
relocate(per_team_id, .after = last_col())

```

Here I'm going to create a variable within each data set that will allow to check for player availability when coding a future outcome variable

Joining the data from the best player by a certain metric data sets to later determine if player was on the court and how that affected the outcome.

```

team_box_scores_2 <- team_box_scores %>%
  left_join(bp_by_ws, by = c('team', 'season'='year')) %>%
  left_join(bp_by_vorp, by = c('team', 'season'='year')) %>%
  left_join(bp_by_per, by = c('team', 'season'='year')) %>%
  mutate(
    ws_tm_game_id = paste0(ws_team_id, '_', gamedate),
    vorp_tm_game_id = paste0(vorp_team_id, '_', gamedate),
    per_tm_game_id = paste0(per_team_id, '_', gamedate))

player_box_scores_2 <- player_box_scores %>%
  left_join(bp_by_ws, by = c('team', 'season'='year', 'name'='ws_name')) %>%
  left_join(bp_by_vorp, by = c('team', 'season'='year', 'name'='vorp_name')) %>%
  left_join(bp_by_per, by = c('team', 'season'='year', 'name'='per_name')) %>%
  mutate(
    ws_plyr_game_id = if_else(
      !is.na(ws_team_id), paste0(ws_team_id, '_', gamedate), NA),
    vorp_plyr_game_id = if_else(
      !is.na(vorp_team_id), paste0(vorp_team_id, '_', gamedate), NA),
    per_plyr_game_id = if_else(
      !is.na(per_team_id), paste0(per_team_id, '_', gamedate), NA))

```

After creating the data sets necessary to determine whether a team's best player (by a given metric) was available I can now create an outcome variable to check for it. I'm also going to add a variable to determines whether a game was a regular season game or playoff game.

Note: *I know I don't have to use multiple mutate functions; however, at times I like to use multiple as it helps me organize my code as I'm writing it.*

```

team_box_scores_3 <- team_box_scores_2 %>%
  mutate(
    ws_bpa = if_else(ws_tm_game_id %in%
      player_box_scores_2$ws_plyr_game_id, 1, 0),
    vorp_bpa = if_else(vorp_tm_game_id %in%
      player_box_scores_2$vorp_plyr_game_id, 1, 0),
    per_bpa = if_else(per_tm_game_id %in%
      player_box_scores_2$per_plyr_game_id, 1, 0)) %>%
  mutate(
    ws_bpa_multi = case_when(
      twolevelloc == 'Loss' & ws_bpa == 0 ~ 0,

```

```

twolevelloc == 'Loss' & ws_bpa == 1 ~ 1,
twolevelloc == 'Win' & ws_bpa == 0 ~ 2,
twolevelloc == 'Win' & ws_bpa == 1 ~ 3),
vorp_bpa_multi = case_when(
  twolevelloc == 'Loss' & vorp_bpa == 0 ~ 0,
  twolevelloc == 'Loss' & vorp_bpa == 1 ~ 1,
  twolevelloc == 'Win' & vorp_bpa == 0 ~ 2,
  twolevelloc == 'Win' & vorp_bpa == 1 ~ 3),
per_bpa_multi = case_when(
  twolevelloc == 'Loss' & per_bpa == 0 ~ 0,
  twolevelloc == 'Loss' & per_bpa == 1 ~ 1,
  twolevelloc == 'Win' & per_bpa == 0 ~ 2,
  twolevelloc == 'Win' & per_bpa == 1 ~ 3)) %>%
mutate(
  ws_bpa_multi = factor(ws_bpa_multi,
    levels = c(0, 1, 2, 3),
    labels = c('Loss Without', 'Loss With',
              'Win Without', 'Win With')),
  vorp_bpa_multi = factor(vorp_bpa_multi,
    levels = c(0, 1, 2, 3),
    labels = c('Loss Without', 'Loss With',
              'Win Without', 'Win With')),
  per_bpa_multi = factor(per_bpa_multi,
    levels = c(0, 1, 2, 3),
    labels = c('Loss Without', 'Loss With',
              'Win Without', 'Win With')))) %>%
mutate(
  gametype = case_when(
    #2021 Season
    (gamedate >= as.Date('2021-05-22') &
     gamedate <= as.Date('2021-07-20')) |
    #2022 Season
    (gamedate >= as.Date('2022-04-16') &
     gamedate <= as.Date('2022-06-16')) |
    #2023 Season
    (gamedate >= as.Date('2023-04-15') &
     gamedate <= as.Date('2023-06-12')) |
    #2024 Season
    (gamedate >= as.Date('2024-04-20') &
     gamedate <= as.Date('2024-06-17')) |
    #2025 Season
    (gamedate >= as.Date('2025-04-19') &
     gamedate <= as.Date('2025-06-2022')) |
    #2026 Season
    (gamedate >= as.Date('2026-04-18') &
     gamedate <= as.Date('2026-06-13')) ~ 'Playoff Game',
    TRUE ~ 'Regular Season Game'))

```

Here I'm going to create some variables that will be used as possible predictors when testing regression models. There are great variables to work with in the data set so far; however, I want to create some rolling averages so I can measure how teams are doing across seasons and within a certain window.

```

regsnbox <- team_box_scores_3 %>%
  filter(gametype == 'Regular Season Game') %>%

```

```

group_by(team) %>%
arrange(desc(gamedate), .by_group=TRUE) %>%
mutate(
  roll_fg_pct = rollmean(fg_pct, k=10, fill=NA, align='left'),
  roll_threefg_pct = rollmean(threefg_pct, k=10, fill=NA, align='left'),
  roll_pts = rollmean(pts, k=10, fill=NA, align='left'),
  roll_three_par = rollmean(three_par, k=10, fill=NA, align='left'),
  roll_dreb_pct = rollmean(dreb_pct, k=10, fill=NA, align='left'),
  roll_ftar = rollmean(ftar, k=10, fill=NA, align='left'),
  roll_ast_pct = rollmean(ast_pct, k=10, fill=NA, align='left'),
  roll_offrating = rollmean(offrating, k=10, fill=NA, align='left'),
  roll_defrating = rollmean(defrating, k=10, fill=NA, align='left'),
  roll_opp_pts = rollmean(opp_pts, k=10, fill=NA, align='left'),
  roll_ptsdiff = rollmean(ptsdiff, k=10, fill=NA, align='left')) %>%
select(gamedate, team, starts_with('roll_'))
pstsbox <- team_box_scores_3 %>%
filter(gametype == 'Playoff Game') %>%
group_by(team) %>%
arrange(desc(gamedate), .by_group=TRUE) %>%
mutate(
  roll_fg_pct = rollmean(fg_pct, k=3, fill=NA, align='left'),
  roll_threefg_pct = rollmean(threefg_pct, k=10, fill=NA, align='left'),
  roll_pts = rollmean(pts, k=3, fill=NA, align='left'),
  roll_three_par = rollmean(three_par, k=3, fill=NA, align='left'),
  roll_dreb_pct = rollmean(dreb_pct, k=3, fill=NA, align='left'),
  roll_ftar = rollmean(ftar, k=3, fill=NA, align='left'),
  roll_ast_pct = rollmean(ast_pct, k=3, fill=NA, align='left'),
  roll_offrating = rollmean(offrating, k=3, fill=NA, align='left'),
  roll_defrating = rollmean(defrating, k=3, fill=NA, align='left'),
  roll_opp_pts = rollmean(opp_pts, k=3, fill=NA, align='left'),
  roll_ptsdiff = rollmean(ptsdiff, k=3, fill=NA, align='left')) %>%
select(gamedate, team, starts_with('roll_'))
fullsbox <- rbind(regsbox, pstsbox)

```

Using the rounding function across across various columns to clean up the rolling averages.

```

team_box_scores_4 <- team_box_scores_3 %>%
left_join(fullsbox, by = c('gamedate', 'team')) %>%
mutate(across(c('roll_fg_pct', 'roll_threefg_pct', 'roll_three_par',
  'roll_ftar'), round, 3)) %>%
mutate(across(c('roll_pts', 'roll_dreb_pct', 'roll_ast_pct',
  'roll_offrating', 'roll_defrating', 'roll_opp_pts',
  'roll_ptsdiff'), round, 1)) %>%
select(-c(ws_name, ws, ws_player_id, ws_team_id, vorp_name, vorp,
  per_name, player_effrtg, player_effrtg_player_id,
  per_team_id, ws_tm_game_id, vorp_tm_game_id,
  per_tm_game_id, vorp_team_id, vorp_player_id))

```

```

## Warning: There was 1 warning in `mutate()`.
## i In argument: `across(...)` .
## Caused by warning:
## ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
## Supply arguments directly to `.fns` through an anonymous function instead.
##

```

```
## # Previously
## across(a:b, mean, na.rm = TRUE)
##
## # Now
## across(a:b, \(x) mean(x, na.rm = TRUE))
```

Summary Data

Adding a non-moving average for the data set, in the off chance I may want to use it for something.

```
team_box_summary <- team_box_scores_4 %>%
  group_by(season, team) %>%
  summarise(
    avg_fg_pct = round(mean(fg_pct, na.rm=TRUE), 3),
    avg_3fg_pct = round(mean(threefg_pct, na.rm=TRUE), 3),
    avg_pts = round(mean(pts, na.rm=TRUE), 1),
    avg_offrating = round(mean(offrating, na.rm=TRUE), 1),
    avg_defrating = round(mean(defrating, na.rm=TRUE), 1))
```

```
## `summarise()` has grouped output by 'season'. You can override using the
## `.groups` argument.
```

Data Export for Visualizations

Export data as RDS files for visualizations in R.

```
setwd('/Users/camsmithers/Desktop/Camalytics/NBA/Data-NBA/Current')
saveRDS(team_box_scores_4, file = 'teamvisdata.rds')
saveRDS(team_box_summary, file = 'teamboxsummary.rds')
```

Export data as CSV files for Python dashboard.

```
setwd('/Users/camsmithers/Desktop/Camalytics/NBA/dashboard')
write_csv(team_box_scores_4, file='teamvisdata.csv')
write_csv(team_box_summary, file='teamboxsummary.csv')
```

Basic Data Dive

```
setwd('/Users/camsmithers/Desktop/Camalytics/NBA')
teamvisdata <- readRDS('Data-NBA/Current/teamvisdata.rds')
teamboxsummary <- readRDS('Data-NBA/Current/teamboxsummary.rds')
data_years <- sort(unique(teamvisdata$season))
```

Here I'm using the `describe` function from the `psych` package to generate a table with summary statistics.

1. N: Number of Observations
2. Mean: Average of the Observations
3. SD: The Standard Deviation
4. Median: The Number in the Middle of the Observations
5. Trimmed: Mean Calculated Without Extreme Values
6. MAD: Mean Absolute Deviation
7. Min: Minimum Value
8. Max: Maximum Value
9. Range: Difference Between Max and Min
10. Skew: Symmetry of the Data Set

11. Kurtosis: Determine the Frequency of Outliers

12. SE: Standard Error

```
describe(teamvisdata)
```

##	vars	n	mean	sd	median	trimmed	mad	min	max
## gamedate	1	15550	NaN	NA	NA	NaN	NA	Inf	-Inf
## team	2	15550	15.41	8.61	15.00	15.40	10.38	1.00	30.00
## opponent	3	15550	15.41	8.61	15.00	15.40	10.38	1.00	30.00
## min	4	15550	241.41	6.41	240.00	240.00	0.00	240.00	315.00
## fg	5	15550	41.45	5.31	41.00	41.37	5.93	22.00	65.00
## fga	6	15550	88.47	7.11	88.00	88.30	7.41	64.00	122.00
## fg_pct	7	15550	0.47	0.06	0.47	0.47	0.06	0.28	0.69
## threefg	8	15550	12.81	3.93	13.00	12.69	4.45	2.00	29.00
## threefga	9	15550	35.55	6.99	35.00	35.40	7.41	10.00	65.00
## threefg_pct	10	15550	0.36	0.08	0.36	0.36	0.08	0.07	0.68
## ft	11	15550	17.46	5.88	17.00	17.25	5.93	0.00	47.00
## fta	12	15550	22.38	7.05	22.00	22.14	7.41	0.00	59.00
## ft_pct	13	15549	0.78	0.10	0.79	0.78	0.10	0.30	1.00
## oreb	14	15550	10.61	3.89	10.00	10.43	4.45	0.00	29.00
## dreb	15	15550	33.24	5.42	33.00	33.16	5.93	14.00	60.00
## treb	16	15550	43.84	6.67	44.00	43.72	7.41	20.00	74.00
## ast	17	15550	25.62	5.19	25.00	25.49	4.45	8.00	50.00
## stl	18	15550	7.73	2.98	7.00	7.60	2.97	0.00	22.00
## blk	19	15550	4.85	2.47	5.00	4.70	2.97	0.00	19.00
## tov	20	15550	13.97	3.93	14.00	13.86	4.45	1.00	31.00
## pf	21	15550	19.44	4.16	19.00	19.36	4.45	4.00	37.00
## pts	22	15550	113.16	12.76	113.00	113.05	13.34	66.00	176.00
## ts_pct	23	15550	0.58	0.06	0.58	0.58	0.06	0.36	0.82
## efg_pct	24	15550	0.54	0.07	0.54	0.54	0.07	0.30	0.81
## three_par	25	15550	0.40	0.07	0.40	0.40	0.08	0.11	0.72
## ftar	26	15550	0.26	0.09	0.25	0.25	0.09	0.00	0.70
## oreb_pct	27	15550	24.01	7.56	23.80	23.86	7.56	0.00	58.80
## dreb_pct	28	15550	75.99	7.56	76.20	76.14	7.56	41.20	100.00
## treb_pct	29	15550	50.00	5.47	50.00	50.00	5.49	29.00	71.00
## ast_pct	30	15550	61.77	9.44	61.90	61.88	9.49	25.70	96.90
## stl_pct	31	15550	7.77	2.92	7.60	7.66	2.97	0.00	21.10
## blk_pct	32	15550	9.16	4.50	8.80	8.93	4.45	0.00	40.40
## tov_pct	33	15550	12.43	3.39	12.30	12.35	3.41	1.00	26.70
## usage_rate	34	15550	100.00	0.00	100.00	100.00	0.00	100.00	100.00
## offrating	35	15550	114.14	11.84	114.20	114.16	12.01	68.30	163.40
## defrating	36	15550	114.14	11.84	114.20	114.16	12.01	68.30	163.40
## homeaway	37	15550	1.50	0.50	1.50	1.50	0.74	1.00	2.00
## season	38	15550	2023.55	1.69	2024.00	2023.56	1.48	2021.00	2026.00
## opp_pts	39	15550	113.16	12.76	113.00	113.05	13.34	66.00	176.00
## ptsdiff	40	15550	0.00	15.54	0.00	0.00	14.83	-73.00	73.00
## twoleveloc	41	15550	1.50	0.50	1.50	1.50	0.74	1.00	2.00
## threeleveloc	42	15550	1.74	0.82	1.50	1.68	0.74	1.00	3.00
## fourleveloc	43	15550	2.50	1.10	2.50	2.50	0.74	1.00	4.00
## ws_bpa	44	15550	0.87	0.34	1.00	0.96	0.00	0.00	1.00
## vorp_bpa	45	15550	0.83	0.38	1.00	0.91	0.00	0.00	1.00
## per_bpa	46	15550	0.74	0.44	1.00	0.80	0.00	0.00	1.00
## ws_bpa_multi	47	15550	2.87	1.08	2.50	2.94	2.22	1.00	4.00
## vorp_bpa_multi	48	15550	2.83	1.11	2.50	2.91	2.22	1.00	4.00
## per_bpa_multi	49	15550	2.74	1.14	2.50	2.80	2.22	1.00	4.00

## gametype	50	15550	1.93	0.25	2.00	2.00	0.00	1.00	2.00
## roll_fg_pct	51	15222	0.47	0.02	0.47	0.47	0.02	0.35	0.60
## roll_threefg_pct	52	15029	0.36	0.03	0.36	0.36	0.03	0.25	0.47
## roll_pts	53	15222	113.17	6.10	113.30	113.29	6.08	85.30	134.30
## roll_three_par	54	15222	0.40	0.05	0.40	0.40	0.05	0.24	0.58
## roll_dreb_pct	55	15222	75.99	3.10	76.10	76.05	2.97	60.00	91.40
## roll_ftar	56	15222	0.26	0.04	0.25	0.25	0.04	0.08	0.49
## roll_ast_pct	57	15222	61.79	4.95	61.70	61.80	4.89	38.40	77.80
## roll_offrating	58	15222	114.18	5.44	114.30	114.23	5.49	86.30	139.70
## roll_defrating	59	15222	114.15	5.19	114.10	114.14	5.04	91.30	136.40
## roll_opp_pts	60	15222	113.15	6.28	113.10	113.15	6.38	85.30	136.50
## roll_ptsdiff	61	15222	0.03	7.20	0.30	0.13	7.12	-36.70	39.70
##	range	skew	kurtosis	se					
## gamedate	-Inf	NA	NA	NA					
## team	29.00	0.00	-1.19	0.07					
## opponent	29.00	0.00	-1.19	0.07					
## min	75.00	5.00	27.87	0.05					
## fg	43.00	0.14	0.01	0.04					
## fga	58.00	0.29	0.36	0.06					
## fg_pct	0.41	0.10	-0.06	0.00					
## threefg	27.00	0.34	0.09	0.03					
## threefga	55.00	0.22	0.04	0.06					
## threefg_pct	0.62	0.08	0.03	0.00					
## ft	47.00	0.38	0.16	0.05					
## fta	59.00	0.37	0.20	0.06					
## ft_pct	0.70	-0.40	0.41	0.00					
## oreb	29.00	0.48	0.33	0.03					
## dreb	46.00	0.16	0.05	0.04					
## treb	54.00	0.21	0.12	0.05					
## ast	42.00	0.26	0.11	0.04					
## stl	22.00	0.44	0.18	0.02					
## blk	19.00	0.64	0.64	0.02					
## tov	30.00	0.28	0.04	0.03					
## pf	33.00	0.19	0.14	0.03					
## pts	110.00	0.10	0.01	0.10					
## ts_pct	0.47	0.11	-0.06	0.00					
## efg_pct	0.51	0.15	-0.03	0.00					
## three_par	0.62	0.11	-0.02	0.00					
## ftar	0.70	0.55	0.48	0.00					
## oreb_pct	58.80	0.21	-0.01	0.06					
## dreb_pct	58.80	-0.21	-0.01	0.06					
## treb_pct	42.00	0.00	0.00	0.04					
## ast_pct	71.20	-0.12	-0.08	0.08					
## stl_pct	21.10	0.39	0.10	0.02					
## blk_pct	40.40	0.59	0.66	0.04					
## tov_pct	25.70	0.25	0.01	0.03					
## usage_rate	0.00	NaN	NaN	0.00					
## offrating	95.10	-0.01	-0.04	0.09					
## defrating	95.10	-0.01	-0.04	0.09					
## homeaway	1.00	0.00	-2.00	0.00					
## season	5.00	-0.02	-1.25	0.01					
## opp_pts	110.00	0.10	0.01	0.10					
## ptsdiff	146.00	0.00	0.31	0.12					
## twoleveloc	1.00	0.00	-2.00	0.00					

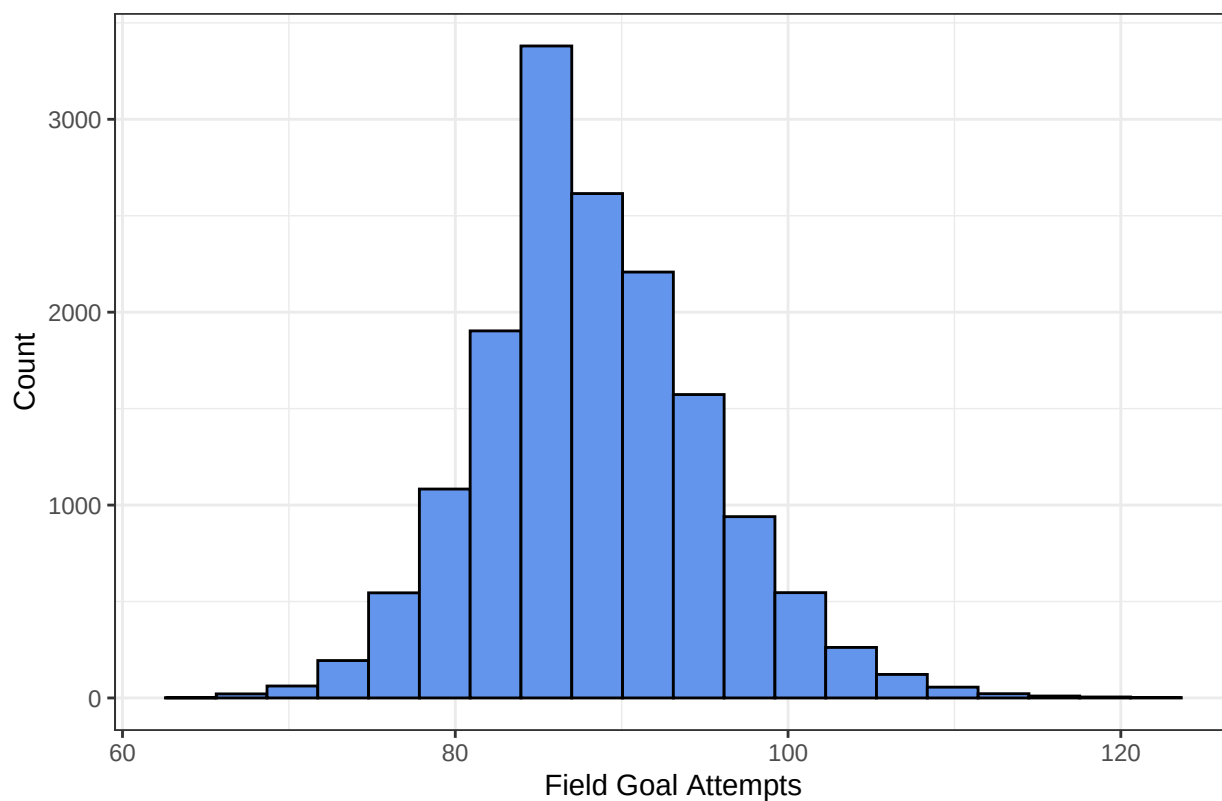

```
## threeleveloc      2.00  0.51   -1.34  0.01
## fourleveloc      3.00  0.00   -1.33  0.01
## ws_bpa           1.00 -2.17    2.69  0.00
## vorp_bpa         1.00 -1.75    1.06  0.00
## per_bpa          1.00 -1.08   -0.84  0.00
## ws_bpa_multi     3.00 -0.12   -1.58  0.01
## vorp_bpa_multi   3.00 -0.13   -1.54  0.01
## per_bpa_multi    3.00 -0.11   -1.50  0.01
## gametype         1.00 -3.53   10.43  0.00
## roll_fg_pct       0.24 -0.04    0.28  0.00
## roll_threefg_pct  0.22  0.03    0.03  0.00
## roll_pts         49.00 -0.20    0.07  0.05
## roll_three_par    0.34  0.19   -0.08  0.00
## roll_dreb_pct     31.40 -0.21    0.62  0.03
## roll_ftar         0.41  0.41    0.82  0.00
## roll_ast_pct     39.40 -0.07    0.16  0.04
## roll_offrating    53.40 -0.10    0.07  0.04
## roll_defrating    45.10 -0.03    0.25  0.04
## roll_opp_pts      51.20 -0.04    0.11  0.05
## roll_ptsdiff      76.40 -0.13    0.15  0.06
```

Now let's create some basic plots to get a better understanding of our data set.

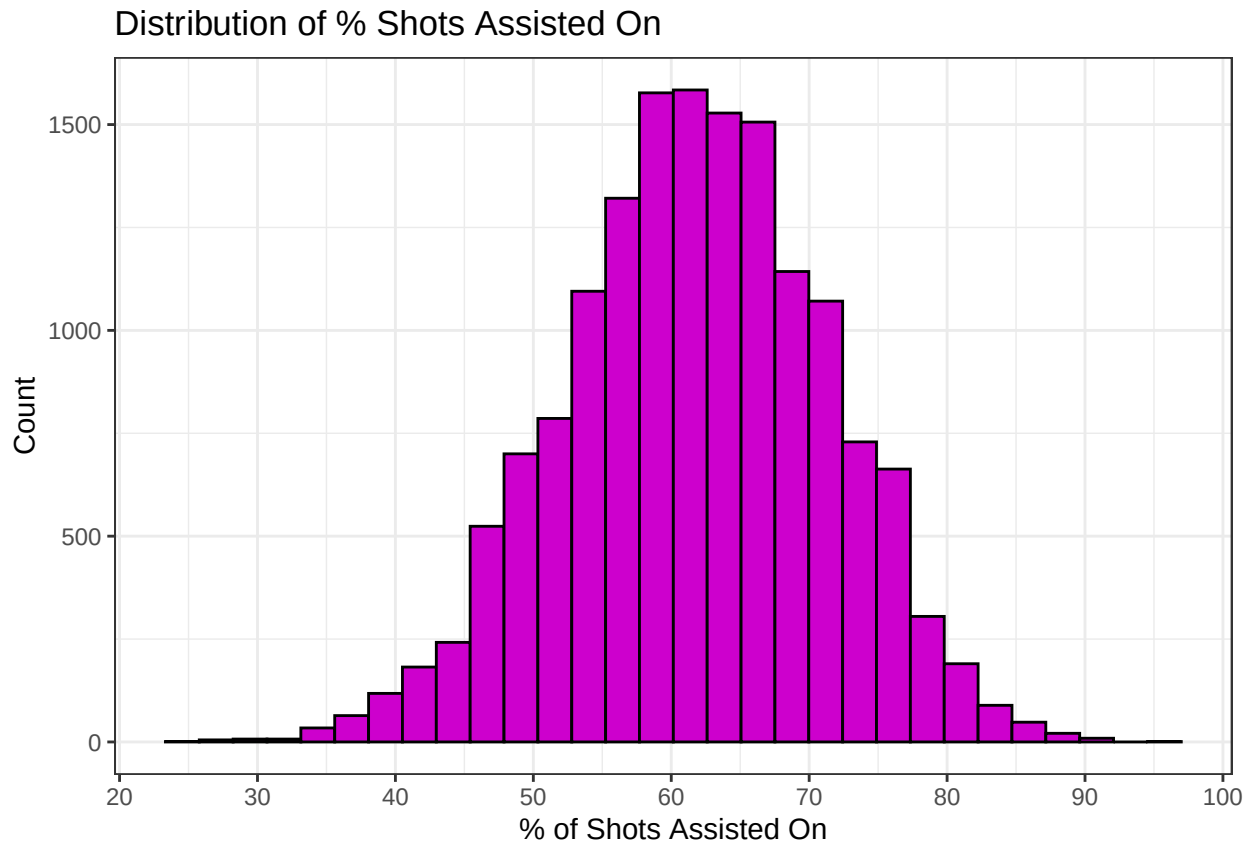
1. Distribution of Field Goal Attempts
2. Distribution of % of Shots Assisted On

```
hist1plot <- ggplot(data=teamvisdata,
                    aes(x=fga)) +
  geom_histogram(bins=20, color='black', fill='cornflowerblue') +
  labs(
    title='Distribution of Field Goal Attempts',
    x='Field Goal Attempts',
    y='Count') +
  theme_bw()
hist1plot
```

Distribution of Field Goal Attempts

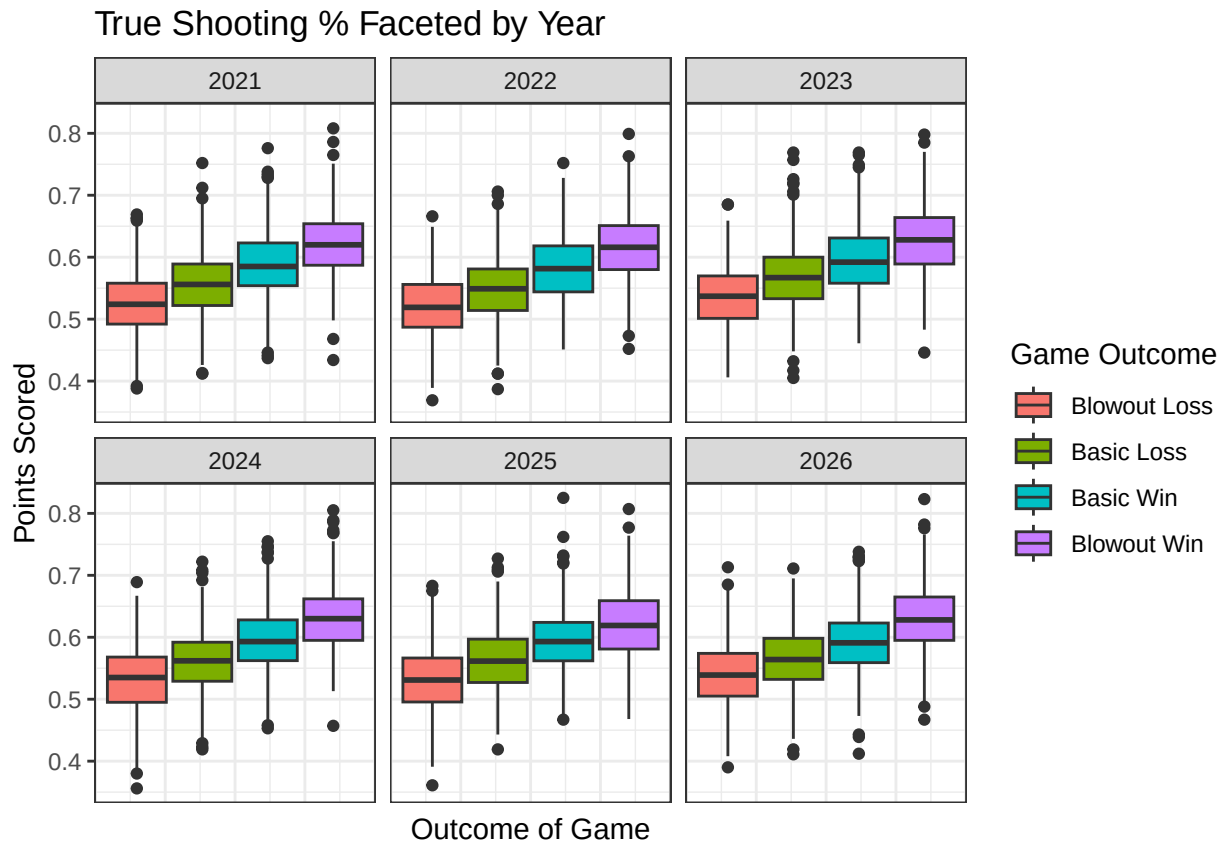


```
hist2plot <- ggplot(data=teamvisdata,
                    aes(x=ast_pct)) +
  geom_histogram(bins=30, color='black', fill='magenta3') +
  labs(
    title='Distribution of % Shots Assisted On',
    x='% of Shots Assisted On',
    y='Count') +
  theme_bw() +
  scale_x_continuous(breaks = seq(from = 0, to = 100, by = 10))
hist2plot
```



Let's look at true shooting percentage based on the outcome of the game (win or loss with magnitude). Let's also facet it by year so we can observe any year to year trends.

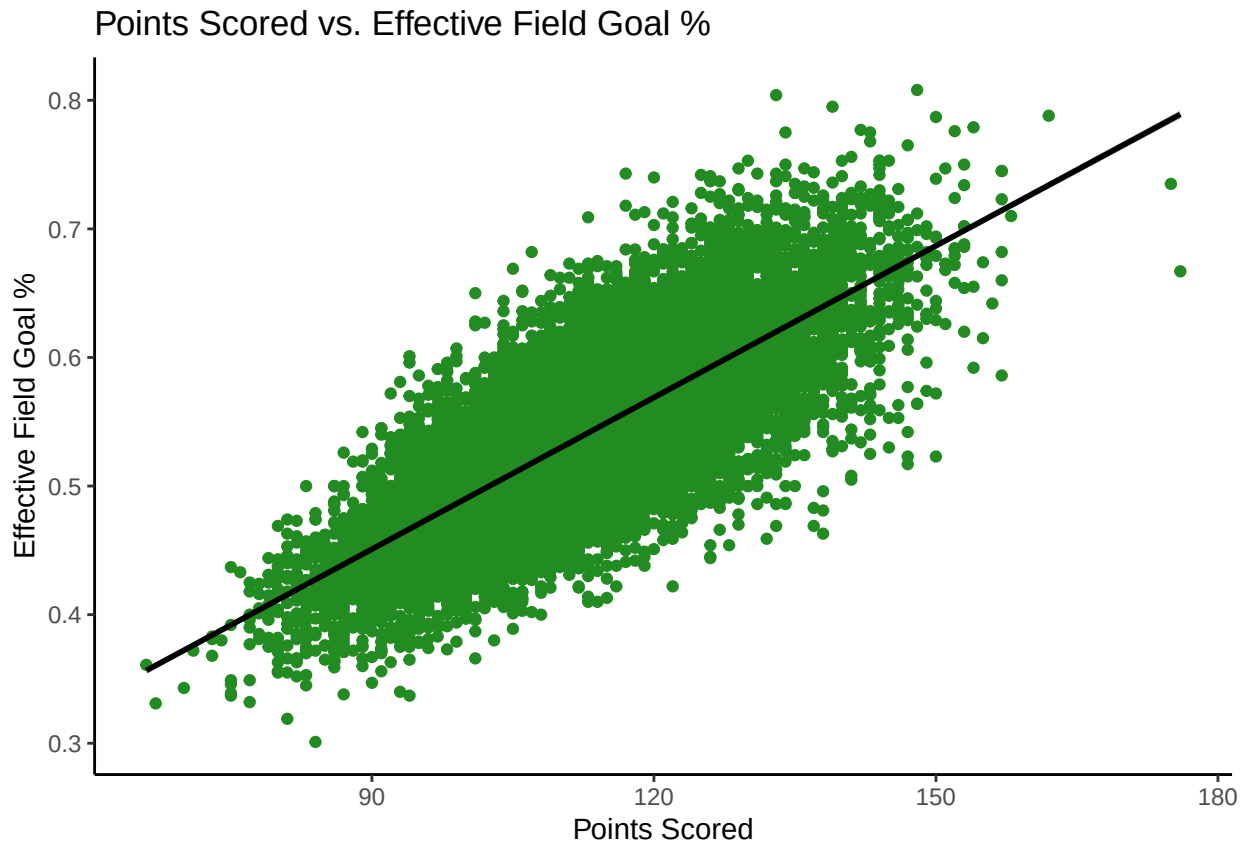
```
boxplot <- ggplot(data=teamvisdata,
                  aes(y=ts_pct,
                      fill=fourleveloc)) +
  geom_boxplot() +
  facet_wrap(~season) +
  labs(
    title='True Shooting % Faceted by Year',
    x='Outcome of Game',
    y='Points Scored',
    fill='Game Outcome') +
  theme_bw() +
  theme(axis.text.x = element_blank(), axis.ticks.x = element_blank())
boxplot
```



I want to look at the relationship between how well teams shoot the ball and how many points they score. I believe there would be a positive relationship between the two, but let's create a plot to double check. Let's also add a regression line with standard error.

```
scatplot <- ggplot(data=teamvisdata,
                   aes(x=pts,
                       y=efg_pct)) +
  geom_point(color='forestgreen') +
  labs(
    title='Points Scored vs. Effective Field Goal %',
    x='Points Scored',
    y='Effective Field Goal %') +
  theme_classic() +
  geom_smooth(method='lm', color='black', se=TRUE)
scatplot
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

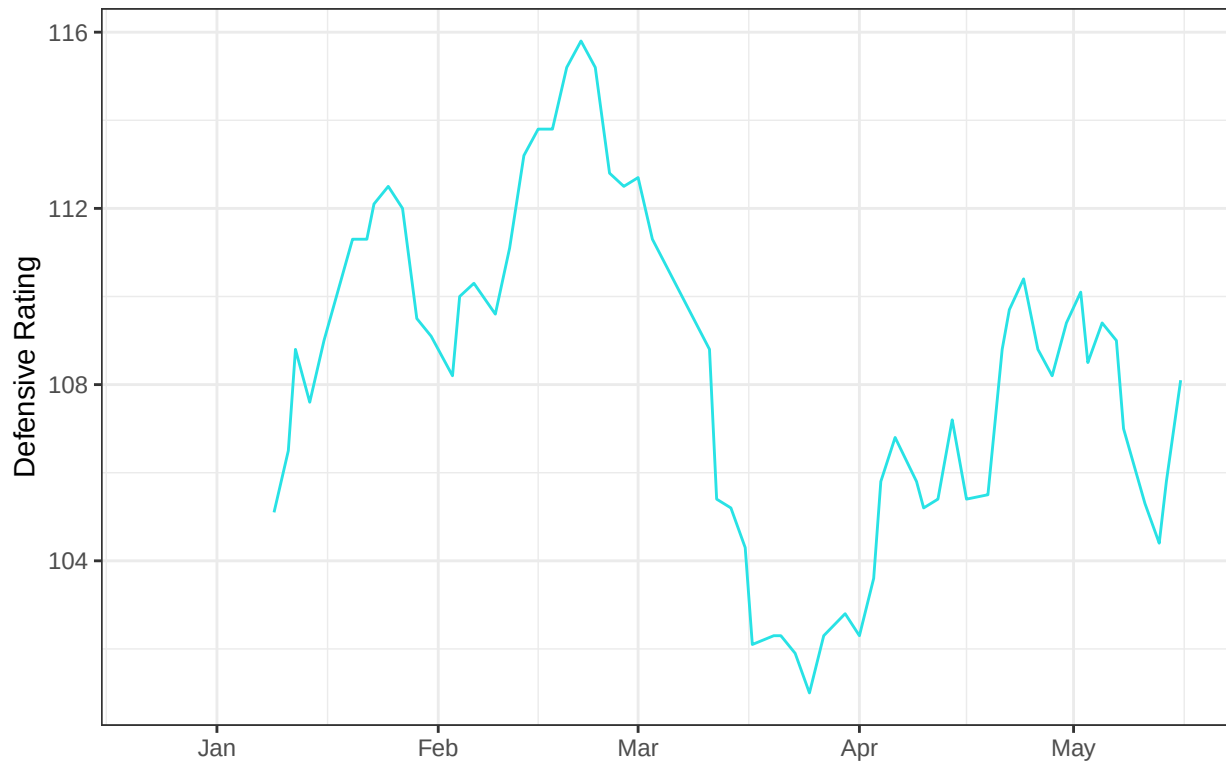


I want to take a look at my favorite team the Philadelphia 76ers, to see how they have performed on the defensive side of the ball over the past few years throughout their seasons.

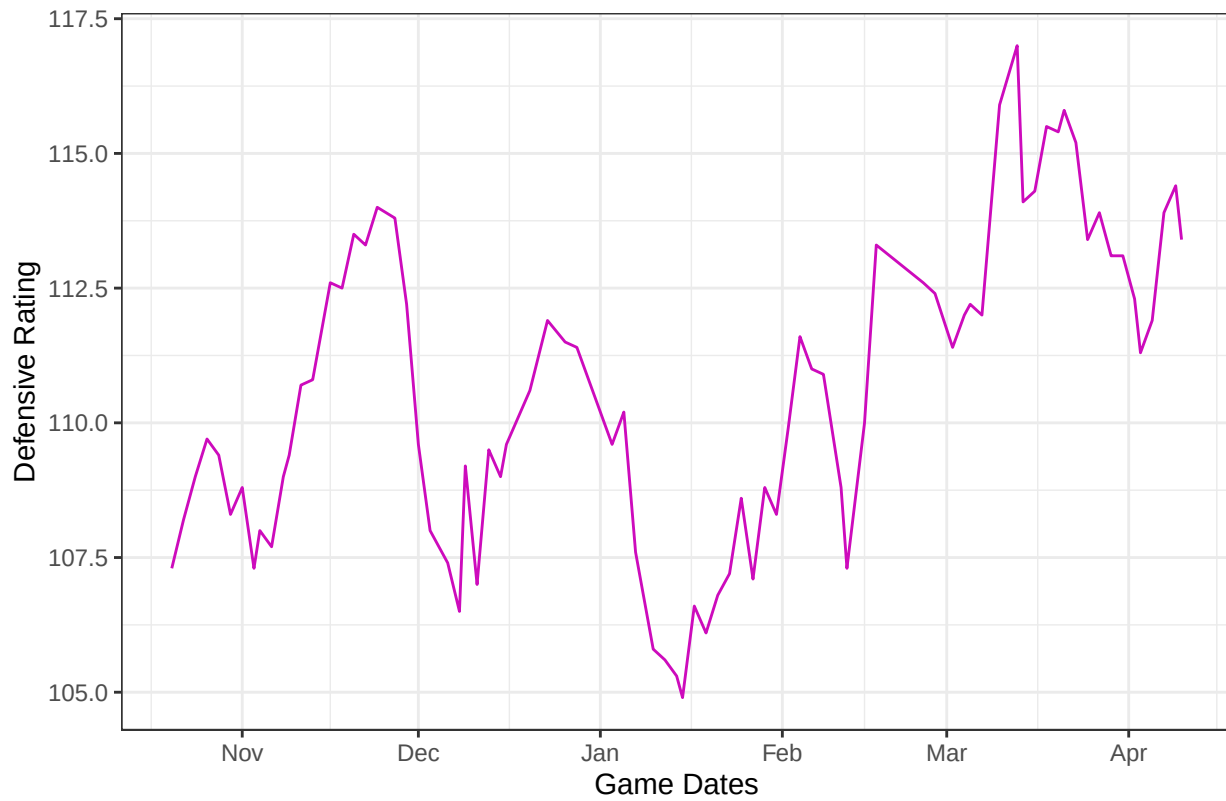
```
for (data_year in data_years) {
  time1plot <- ggplot(data=teamvisdata %>%
    filter(team=='PHI',
           season==data_year,
           gametype=='Regular Season Game'),
    aes(x=gamedate,
        y=roll_defrating)) +
  #   geom_point() +
  geom_line(color=data_year) +
  theme_bw() +
  scale_x_date(date_breaks='1 month', date_labels='%b') +
  labs(
    title=glue('{data_year} Defensive Rating (Moving Average (n=10))'),
    x='Game Dates',
    y='Defensive Rating'
  )
  print(time1plot)
}
```

```
## Warning: Removed 9 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

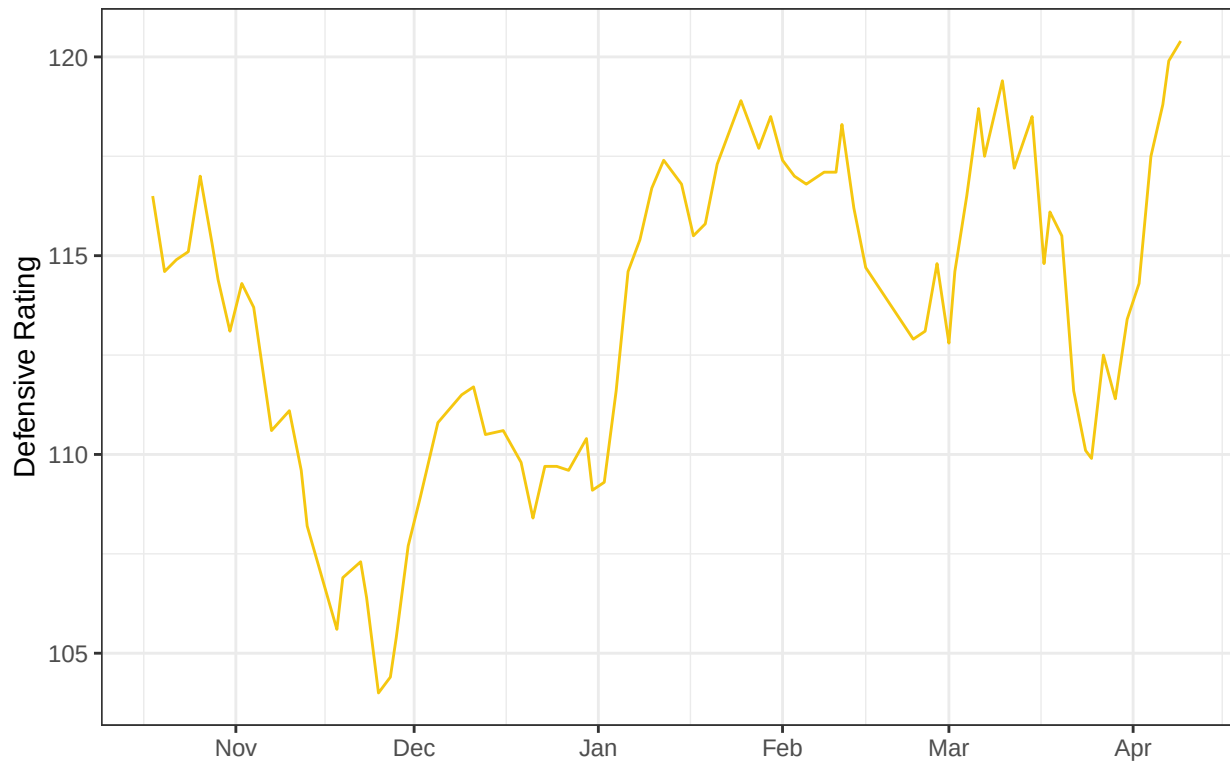
2021 Defensive Rating (Moving Average (n=10))



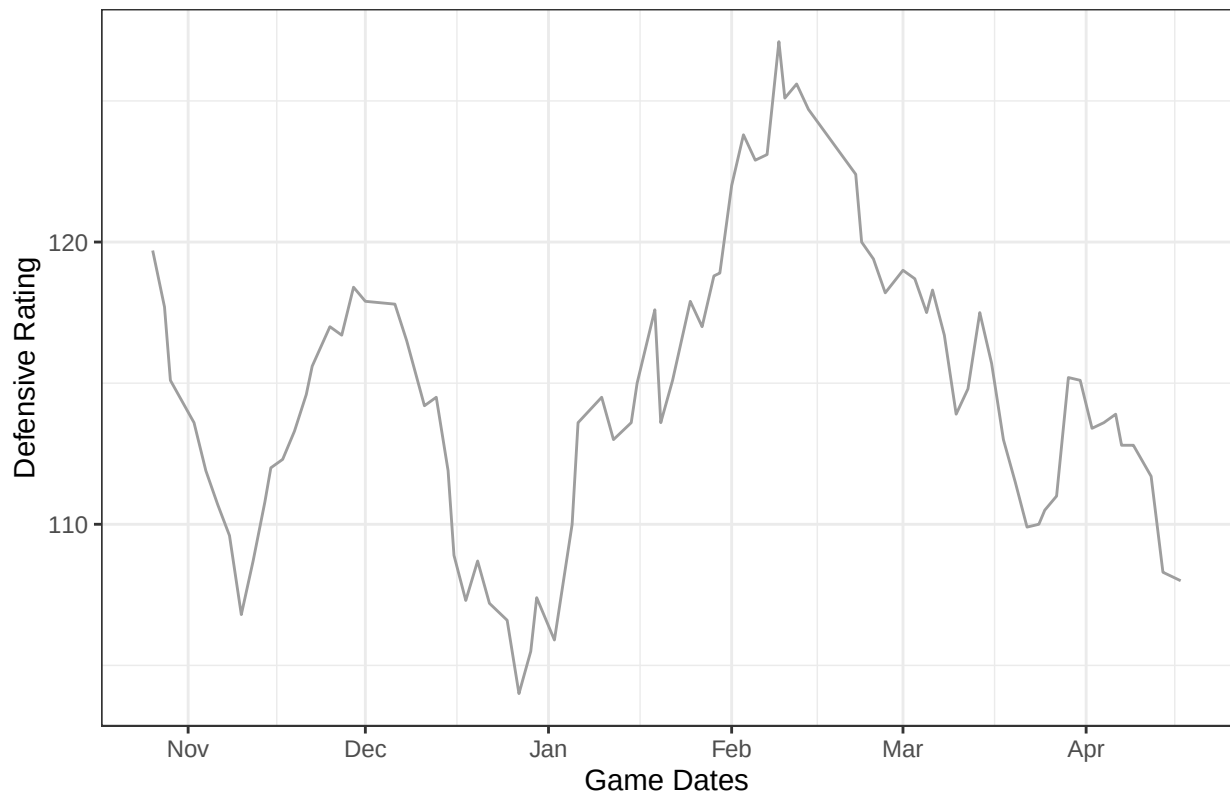
2022 Defensive Rating (Moving Average (n=10))

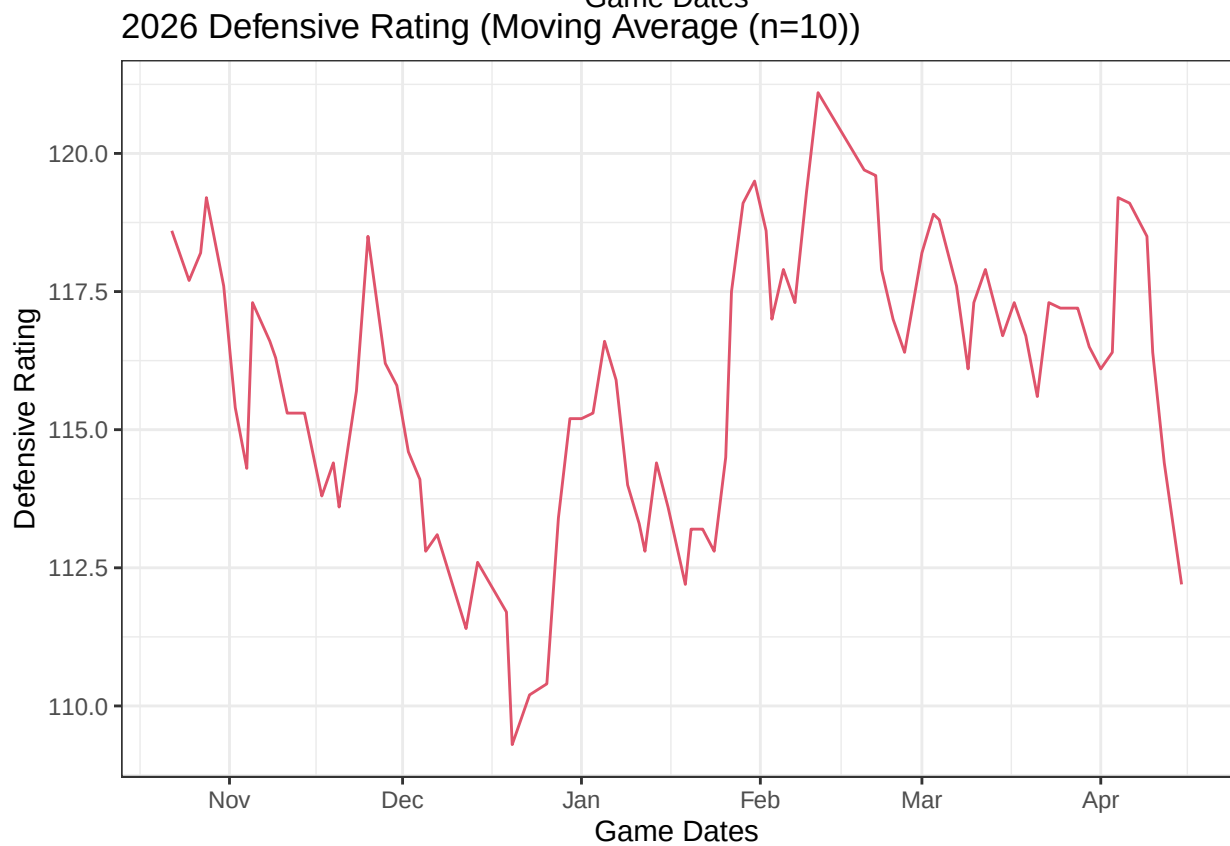
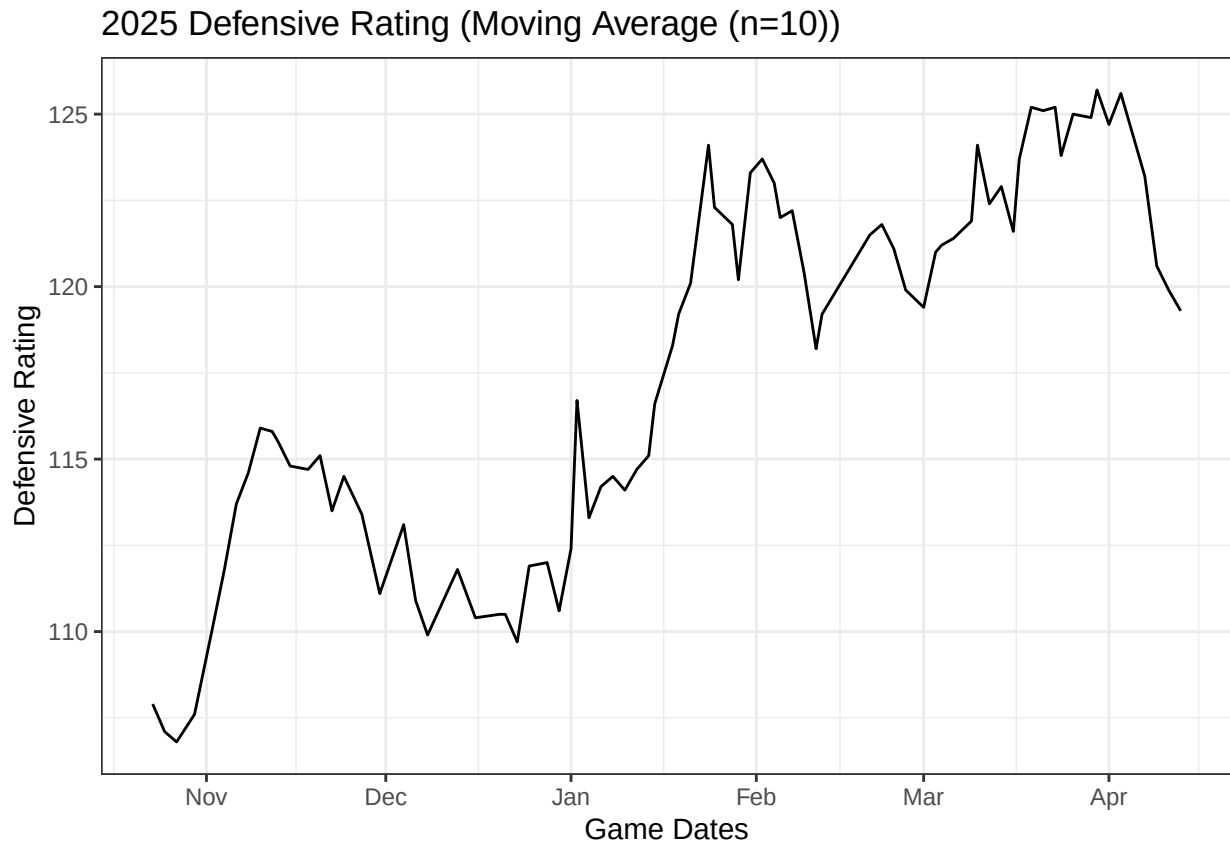


2023 Defensive Rating (Moving Average (n=10))



2024 Defensive Rating (Moving Average (n=10))





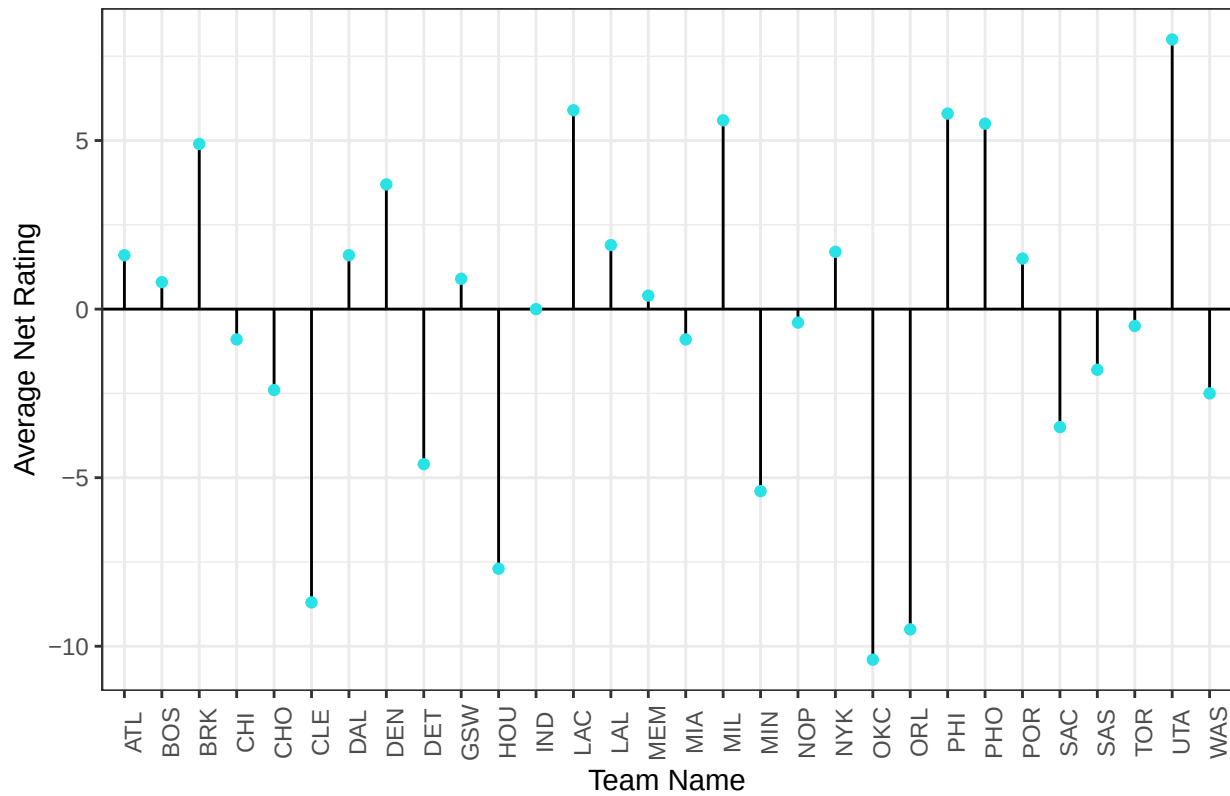
Let's create a lollipop chart to look at teams average net rating for each season in the data set.

- Net Rating = Offensive Rating - Defensive Rating
 - Rating: Looks at how many points a teams scores or allows per 100 possessions. This allows to compare teams between games that may have had significantly different paces of play.

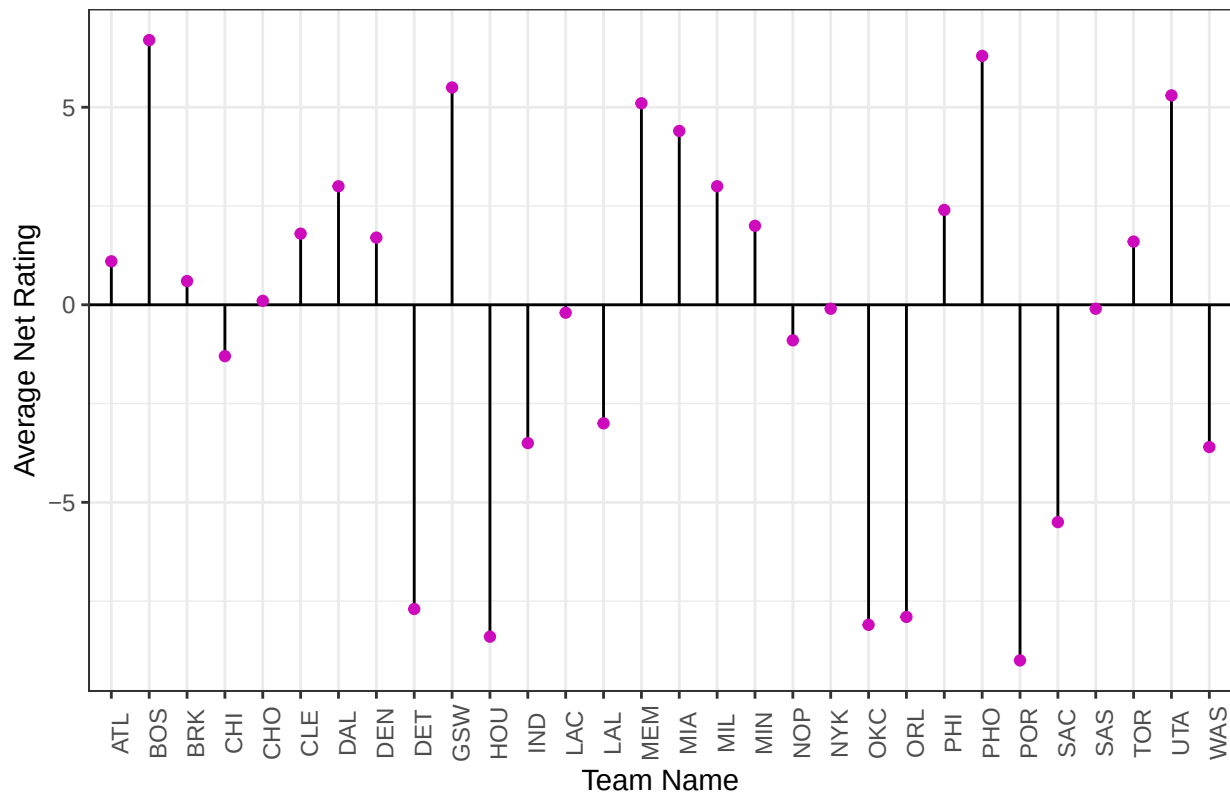
```
data_years <- sort(unique(teamvisdata$season))

for (data_year in data_years) {
  lol1plot <- ggplot(data=teamboxsummary %>%
    mutate(avg_netrating =
      avg_offrating - avg_defrating) %>%
    filter(season==data_year),
    aes(x=team, y=avg_netrating)) +
  geom_segment(aes(
    x=team,
    xend=team,
    y=0,
    yend=avg_netrating), color='black') +
  theme_bw() +
  geom_hline(yintercept=0) +
  theme(axis.text.x = element_text(angle = 90)) +
  geom_point(color=data_year) +
  labs(
    title=glue('{data_year} Average Net Rating By Team'),
    x='Team Name',
    y='Average Net Rating'
  )
  print(lol1plot)
}
```

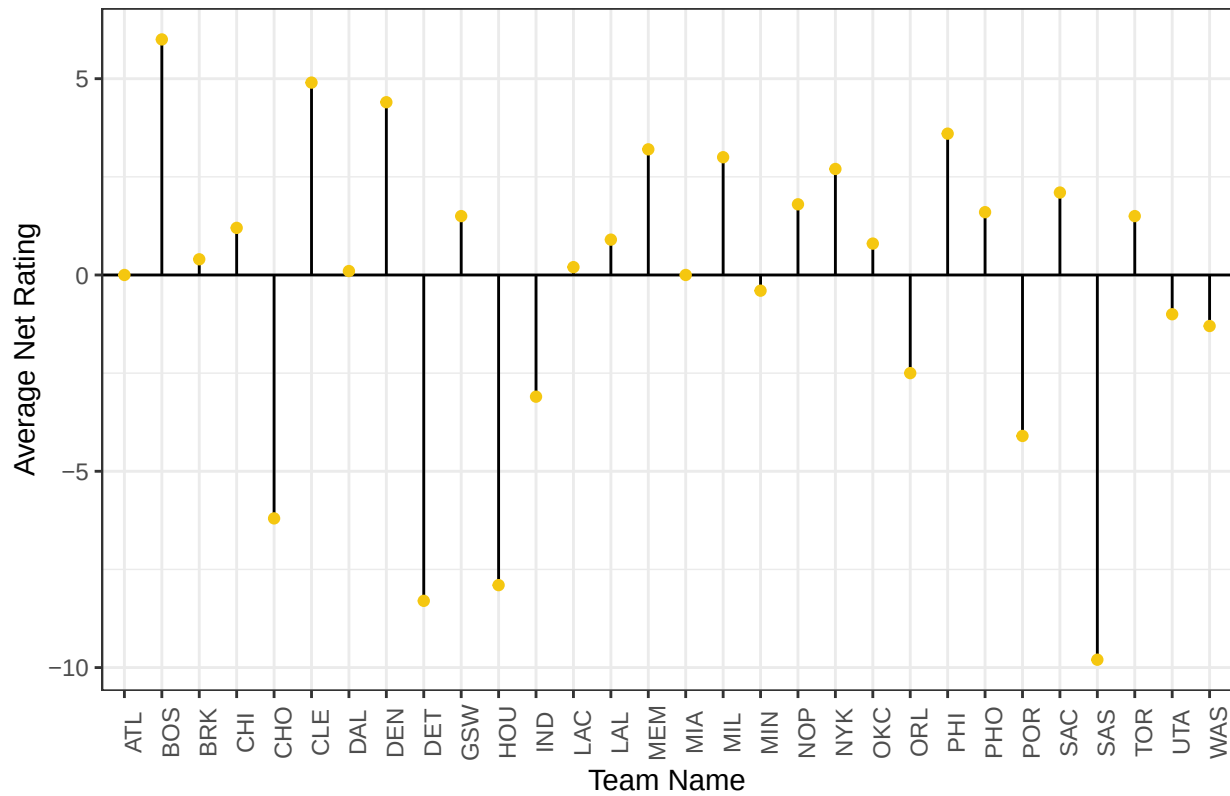
2021 Average Net Rating By Team



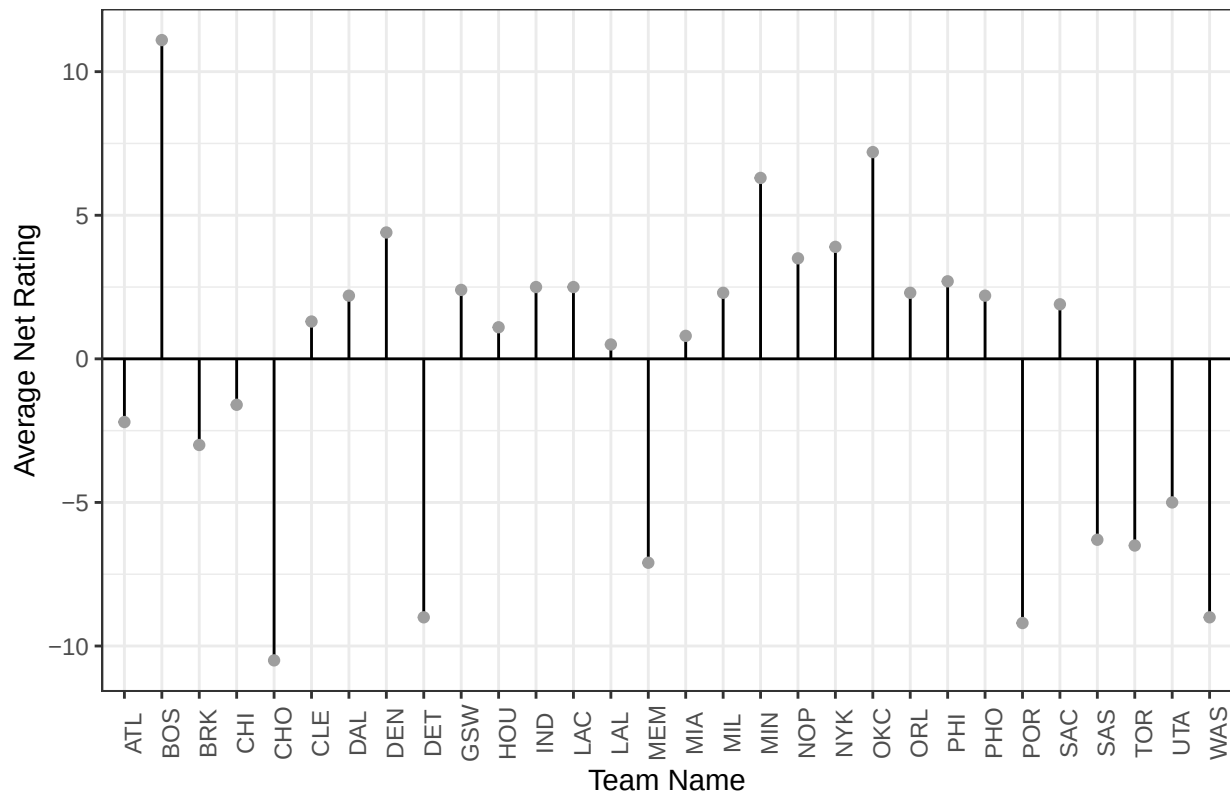
2022 Average Net Rating By Team



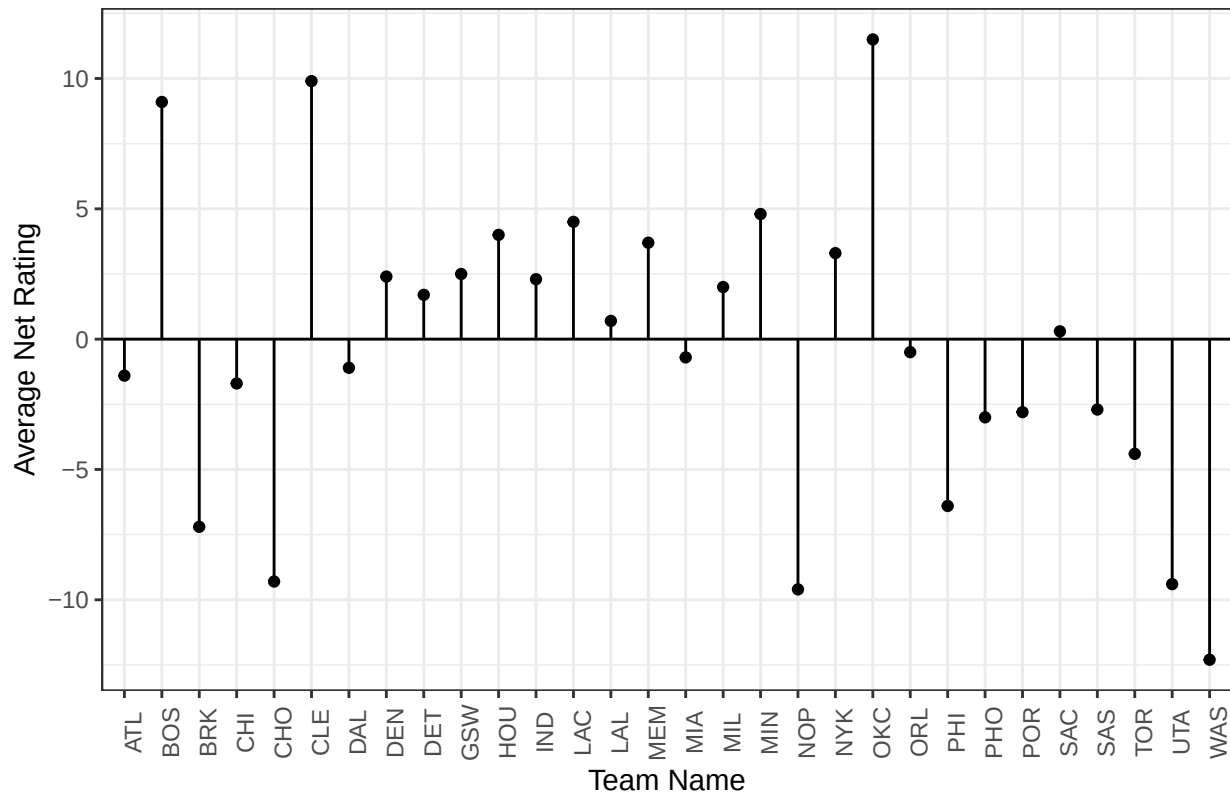
2023 Average Net Rating By Team



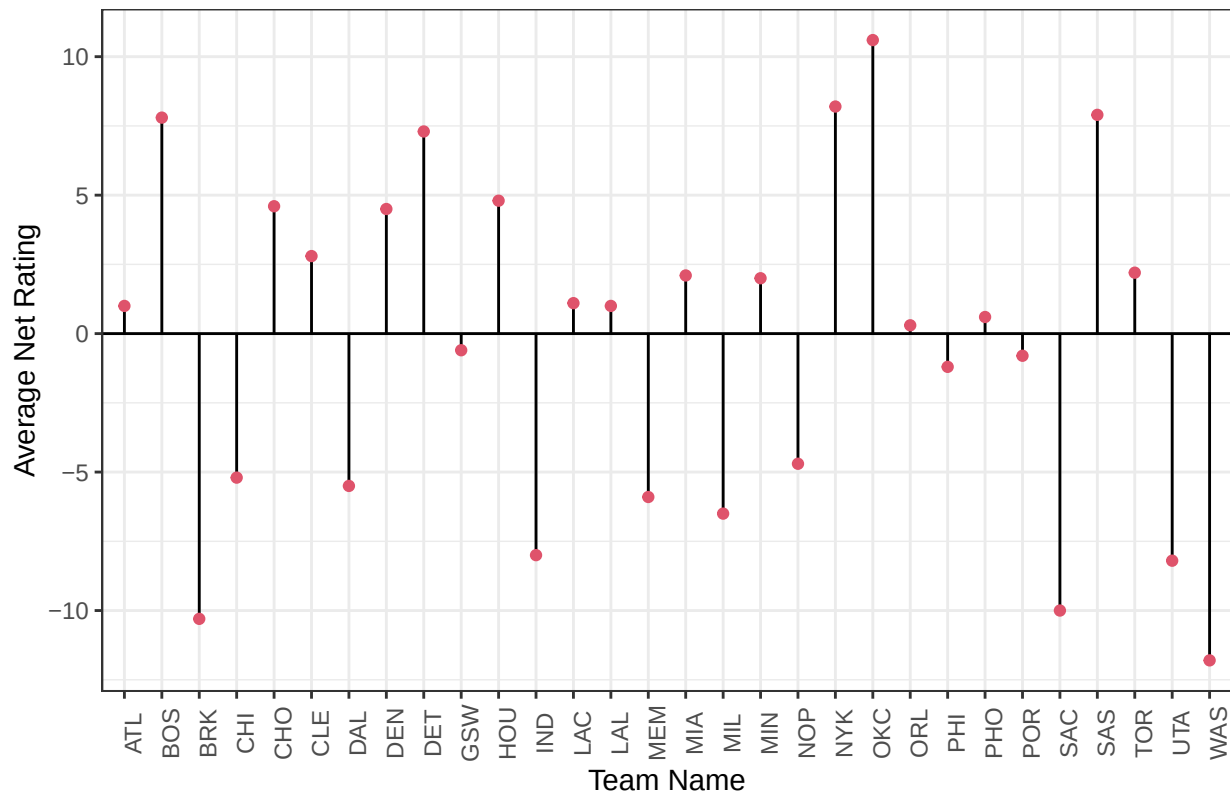
2024 Average Net Rating By Team



2025 Average Net Rating By Team



2026 Average Net Rating By Team



Advanced Data Dive

Here I'm going to load in the data we've been working with and do some slight cleaning up of our variables prior to running some regression models.

```
setwd('/Users/camsmithers/Desktop/Camalytics/NBA')
teamvisdata <- readRDS('Data-NBA/Current/teamvisdata.rds') %>%
  na.omit() %>%
  mutate(team = as.factor(team),
         opponent = as.factor(opponent),
         season = as.factor(season)) %>%
  arrange(gamedate)
teamboxsummary <- readRDS('Data-NBA/Current/teamboxsummary.rds')
data_years <- sort(unique(teamvisdata$season))
```

Let's run the model that was the best performing during cross validation. Then let's add the predicted values from the model to our data set to use in some visualizations.

```
winbinary1 <- glm(twoleveloc ~ roll_offrating + roll_defrating + team +
                 homeaway + vorp_bpa + roll_three_par + roll_ftar +
                 roll_ast_pct,
                 data=teamvisdata, family='binomial')

winbinary1_summary <- summary(winbinary1)

winbinary1_fitted <- fitted(winbinary1)

winbinary1_approx <- predict(winbinary1, newdata=teamvisdata, type='response')

teamvisdata2 <- cbind(teamvisdata, winbinary1_approx) %>%
  mutate(winbinary1_approx = round(100 * winbinary1_approx, 1))

testmodel <- multinom(fourleveloc ~ 1, data = teamvisdata)
```

```
## # weights:  8 (3 variable)
## initial  value 20833.231659
## final   value 20826.546265
## converged
```

```
summary(testmodel)
```

```
## Call:
## multinom(formula = fourleveloc ~ 1, data = teamvisdata)
##
## Coefficients:
##              (Intercept)
## Basic Loss    0.0586211519
## Basic Win     0.0612057979
## Blowout Win  0.0005490061
##
## Std. Errors:
##              (Intercept)
## Basic Loss    0.02309162
## Basic Win     0.02307715
## Blowout Win   0.02342428
##
## Residual Deviance: 41653.09
```

AIC: 41659.09